

Node Inicializr

Documentacao tecnica consolidada - Sprints 1 e 2

Item	Descricao
Produto	Gerador full-stack para projetos Node.js a partir de GenerateConfig validado.
Sprint 1	Fundacao do monorepo, contratos compartilhados, API base, web app, validacao e qualidade.
Sprint 2	Motor de geracao: resolver, composers, templates reais, zip, rota HTTP e smoke test.
Entregavel atual	POST /api/generate gera um .zip funcional para Express + TypeScript + Prisma + JWT.
Validacao final	pnpm format:check, pnpm lint, pnpm build, pnpm test, npm install, npm run dev e npm run build no projeto gerado.

Objetivo do documento

Explicar o que foi construido, por que cada artefato existe, como as partes se conectam e como validar o projeto. A leitura deve permitir que uma pessoa nova continue a Sprint 3 sem depender do historico da implementacao.

1. Visao Geral

O Node Initializr e um monorepo pnpm/Turbo com API, Web e pacote shared. A API recebe uma configuracao estruturada, valida combinacoes de tecnologias, resolve fragmentos de template, compoe arquivos e retorna um projeto Node.js compactado em zip.

Camada	Responsabilidade	Artefatos
packages/shared	Contrato de configuracao, tipos e compatibilidade reutilizaveis.	schema.ts, types.ts, compatibility.ts
apps/api/routes	Health check e endpoint POST /api/generate com erro padronizado.	health.route.ts, generate.ts, errorHandler.ts
apps/api/engine	Validar, resolver, renderizar, compor package/infra e zipar.	validator.ts, resolver.ts, index.ts, composers/*
apps/api/templates	Templates EJS reais do caminho feliz.	base, express, prisma, jwt
apps/web	Base React/Vite com Tailwind, shadcn/Radix e validacoes de compatibilidade.	App.tsx, compatibility.ts, button.tsx

Fluxo de geracao

```
POST /api/generate
-> express.json()
-> validate(payload)
-> resolveFragments(config)
-> renderFiles(fragments with files/)
-> composePackageJson(config, fragments)
-> composeInfraFiles(config)
-> ZipArchive(files)
-> response application/zip
```

2. Sprint 1 - Fundacao

A Sprint 1 estabeleceu o contrato e a base operacional. O ponto mais importante foi centralizar schema, tipos e compatibilidade no pacote shared para evitar divergencia entre API e Web.

Area	O que foi feito	Por que importa
Monorepo	package.json raiz, pnpm-workspace.yaml, turbo.json e scripts padronizados.	Comandos unificados para api, web e shared.
Qualidade	ESLint flat config, Prettier, build, lint, test e format check.	Barreira automatizada contra regressao.
Shared	Schemas Zod, tipos GenerateConfig/ValidationError e compatibility.	Fonte unica do contrato.
API base	Express, app/server, health route, Vitest e Supertest.	Primeira superficie HTTP testavel.
Web base	Vite React TS, Tailwind v4, shadcn/Radix, Vitest e Testing Library.	Base para interface de configuracao.
Validator	validate(payload) com envelope de erro e INVALID_COMBINATION.	Impede que configuracao invalida chegue ao motor.
Compatibilidade Web	lib/compatibility.ts e testes.	Permite feedback antecipado na UI.

Contrato GenerateConfig

```
{
  "name": "meu-projeto",
  "packageManager": "npm|pnpm|yarn",
  "framework": "express|fastify|nestjs|hono",
  "language": "typescript|javascript",
  "architecture": "modular|clean|mvc",
  "database": "postgresql|mysql|mongodb|sqlite|none",
  "orm": "prisma|typeorm|drizzle|mongoose|none",
  "auth": "jwt|clerk|none",
  "messaging": "none|rabbitmq|bullmq",
  "dependencies": ["redis", "swagger", "docker", ...]
}
```

Ajustes relevantes

- Campo packageManager no package.json raiz para o Turbo resolver workspaces.
- Imports ESM com extensao .js em TypeScript NodeNext/Node16.
- tsconfigRootDir para evitar falsos erros do ESLint no editor.
- Aliases para shadcn/Radix e dependencias de UI corrigidas.

3. Sprint 2 - Motor de Geracao

A Sprint 2 implementou o pipeline completo. O motor recebe um GenerateConfig valido e produz um Buffer zip com arquivos renderizados, package.json composto e infraestrutura opcional.

Modulo	Responsabilidade	Detalhes
PackageComposer	Compor package.json final.	Agrega deps/devDeps, resolve maior semver, remove ^/~, preserva script por prioridade e gera scripts por linguagem.
FileRenderer	Renderizar templates EJS.	Percorre files/, injeta config, remove .ejs, bloqueia path traversal, consolida .env.example e detecta colisoes.
InfraComposer	Gerar Dockerfile e docker-compose.yml.	Ativado por docker; suporta banco e Redis conforme config.
Resolver	Selecionar fragmentos.	Mapeia base, framework, ORM, auth e dependencias para registry ordenado.
Engine index	Orquestrar o motor.	Valida, resolve, renderiza, compoe package/infra e compacta com ZipArchive.
POST /api/generate	Expor geracao HTTP.	Retorna application/zip, Content-Disposition, Cache-Control no-store, 400 e 500 padronizados.
ErrorHandler	Centralizar erro interno.	formatError(error) e middleware global Express com teste unitario.

Templates reais

Template	Arquivos	Manifest/dependencias
base/typescript	tsconfig.json, .gitignore, .env.example	typescript 5.5.4, tsx 4.19.2, scripts dev/build/start, PORT.
base/javascript	.gitignore, .env.example	PORT e estrutura minima.
frameworks/express	src/app, src/server, src/routes/index em TS e JS	express 4.19.2, @types/express 4.17.21.
orms/prisma	schema.prisma, src/lib/prisma em TS e JS	@prisma/client, prisma, db:migrate, db:generate, DATABASE_URL.
auth/jwt	src/lib/jwt, src/middlewares/auth em TS e JS	jsonwebtoken, @types/jsonwebtoken, JWT_SECRET, JWT_EXPIRES_IN.

Contrato HTTP

```
POST /api/generate
Sucesso: 200 application/zip + Content-Disposition + Cache-Control: no-store
Validacao: 400 { "error": { "code", "message", "field", "details" } }
Interno: 500 { "error": { "code": "GENERATION_FAILED", "message": "Falha ao gerar projeto." } }
```

4. Decisoese de Arquitetura

O motor nao conhece HTTP e os composers nao escrevem em disco. Essa separacao permite testar o zip com JSZip, executar smoke test real e evoluir templates sem alterar a rota publica.

Responsabilidade	Local	Racional
Validacao	shared/schema.ts + engine/validator.ts	Zod centraliza regras e validator transforma issues em erro do produto.
Selecao	resolver.ts + fragmentRegistry.ts	Tecnologias viram fragments extensivos por prioridade.
Package	packageComposer.ts	Evita inconsistencias de dependencias e scripts.
Arquivos	fileRenderer.ts	Templates declarativos com protecoes contra path traversal.
Infra	infraComposer.ts	Docker apenas quando solicitado.
HTTP	routes/generate.ts	Headers corretos e stack trace oculto.

Ajustes descobertos no smoke test

Problema	Causa	Correcao
Zip de 74 bytes	Arquivo salvo era JSON de erro, nao zip.	Inspecao por tamanho/assinatura e uso de payload.json com curl --data-binary.
shared falhava em runtime	exports sem condicao default.	Adicionado default em packages/shared/package.json.
npm run dev falhava	base/typescript no registry sem manifest; faltavam tsx/typescript.	Manifest corrigido e teste de regressao adicionado.
Porta 3000 presa	Processo Node antigo/orfao.	Smoke test em API limpa na 3001 e projeto na 4000.

5. Catalogo de Artefatos

Resumo dos artefatos criados ou consolidados e o motivo da existencia de cada um.

Artefato	O que e	Por que existe
package.json raiz	Scripts Turbo e devDependencies globais.	Centraliza comandos do monorepo.
turbo.json	Pipeline build/lint/test/dev.	Ordena tasks e cacheia quando aplicavel.
shared/schema.ts	Schema Zod do GenerateConfig.	Fonte unica do contrato.
shared/compatibility.ts	Regras de combinacao.	Reuso por backend e frontend.
validator.ts	Adaptador para ValidationResult.	Erro padronizado para rota e motor.
resolver.ts	Resolve fragments por config.	Transforma escolhas em partes de projeto.
fragmentRegistry.ts	Registro de fragments, prioridades e manifests.	Ponto de extensao para novas tecnologias.
packageComposer.ts	Composicao do package.json.	Une dependencias/scripts.
fileRenderer.ts	Renderizacao EJS.	Gera arquivos reais.
infraComposer.ts	Dockerfile e docker-compose.	Infra quando docker e selecionado.
engine/index.ts	Orquestrador e zip.	Conecta todos os componentes.
routes/generate.ts	Endpoint de download.	Superficie HTTP do motor.
errorHandler.ts	formatError e middleware global.	Padroniza falhas internas.
templates/*	Templates EJS reais.	Materializam o caminho feliz.
apps/web/*	Frontend base.	Interface futura de configuracao.

6. Testes e Validacao

A cobertura acompanha o risco: contratos e health na Sprint 1; composers, zip real, rota HTTP e smoke externo na Sprint 2.

Suite	Cobertura	Resultado
packages/shared	Schema e compatibilidade.	2 arquivos, 6 testes passando.
apps/api engine	Resolver, validator, package/file/infra composers e orchestrator.	9 arquivos de teste API, 125 testes passando.
apps/api routes	Health, generate, erros internos e contrato HTTP.	Coberto por Supertest.
apps/web	Render basico e compatibilidade frontend.	2 arquivos, 9 testes passando.
Smoke externo	Zip via HTTP, npm install, npm run dev, health e npm run build.	Passou para Express + TS + Prisma + JWT.

Comandos finais

```
pnpm format:check
pnpm lint
pnpm build
pnpm test

# Projeto gerado
npm install
npm run dev # validado com PORT=4000
curl http://localhost:4000/api/health
npm run build
```

Evidencia do smoke test

Etapas	Evidencia
Geracao HTTP	Zip em C:\Users\User\Desktop\generated-smoke\generated-project.zip.
Assinatura ZIP	504b0304.
Tamanho final	3938 bytes apos correcao do manifest TypeScript.
Arquivos extraidos	package.json, tsconfig.json, Dockerfile, docker-compose.yml, src/app.ts, src/server.ts, prisma/schema.prisma, JWT e Prisma libs.
Health	HTTP/1.1 200 OK com {"status":"ok"}.
Build	npm run build executou tsc sem erro.

7. Guia Operacional

No PowerShell, grave o JSON em arquivo e envie com --data-binary para evitar problemas de escape com aspas e arrays no curl.exe.

```
cd C:\Users\User\Desktop\generated-smoke

$payload = @{
    name = "meu-projeto"
    packageManager = "npm"
    framework = "express"
    language = "typescript"
    architecture = "modular"
    database = "postgresql"
    orm = "prisma"
    auth = "jwt"
    messaging = "none"
    dependencies = @("docker", "swagger")
} | ConvertTo-Json -Compress

Set-Content .\payload.json $payload -NoNewline -Encoding utf8
curl.exe -X POST "http://localhost:3000/api/generate" `
  -H "Content-Type: application/json" `
  --data-binary "@payload.json" `
  --output "generated-project.zip"
```

Como reconhecer erro vs zip

- Zip valido começa com 504b0304.
- Arquivo de 74 bytes nesse caso era JSON de erro GENERATION_FAILED.
- Se o Windows disser que a pasta compactada e invalida, leia o arquivo como texto.
- Se a porta 3000 estiver presa, use netstat -ano | findstr :3000 ou rode temporariamente com PORT=3001.

Package gerado no caminho feliz

```
{
  "scripts": { "dev": "tsx watch src/server.ts", "build": "tsc" },
  "dependencies": { "express": "4.19.2", "@prisma/client": "5.14.0", "jsonwebtoken": "9.0.2" },
  "devDependencies": { "typescript": "5.5.4", "tsx": "4.19.2", "@types/express": "4.17.21", "prisma": "5.14.0", "@types"
}
```

8. Estado Atual e Proximos Passos

Ao fim da Sprint 2, o caminho feliz definido pelo plano esta funcional: a API gera um zip real para Express + TypeScript + Prisma + JWT, o projeto extraido instala, sobe em modo dev, responde health check e compila.

Item	Status
POST /api/generate retorna zip valido	Concluido.
Projeto Express + TS + Prisma + JWT gerado	Concluido.
npm install no projeto gerado	Concluido.
npm run dev no projeto gerado	Concluido com PORT=4000.
npm run build no projeto gerado	Concluido.
pnpm format:check/lint/build/test no monorepo	Concluido.

Riscos conhecidos

- Geracao de zip ainda usa Buffer em memoria; stream direto pode reduzir consumo futuro.
- Registry ainda mistura manifests inline e manifest.json; carregar do disco reduziria duplicacao.
- Fragments nao implementados ainda apontam para fixtures vazias.
- npm audit do projeto gerado indicou vulnerabilidades; deve entrar no backlog tecnico.
- Existe template com nome aparente jwt.js.ejs em auth/jwt; revisar antes de ampliar suporte JavaScript/JWT.

Sugestoes para Sprint 3

- Conectar frontend ao POST /api/generate e implementar download do zip.
- Adicionar feedback visual de validacao e incompatibilidades.
- Automatizar smoke test de gerar, extrair, instalar e compilar.
- Expandir templates para outras combinacoes do schema.